

A top-down view of various school supplies arranged on a white surface. In the top left, a black mesh pen holder contains several pens and pencils. Below it, a pair of blue-handled scissors is open. To the right of the scissors is a red pencil, a white eraser, and a small blue sharpener. Further right is a black calculator with a small screen and various function buttons. Next to the calculator is a blue stapler. At the bottom, a long blue ruler with white markings is laid out. Above the ruler, there are two blue protractors, a pink sticky note, and a silver pen.

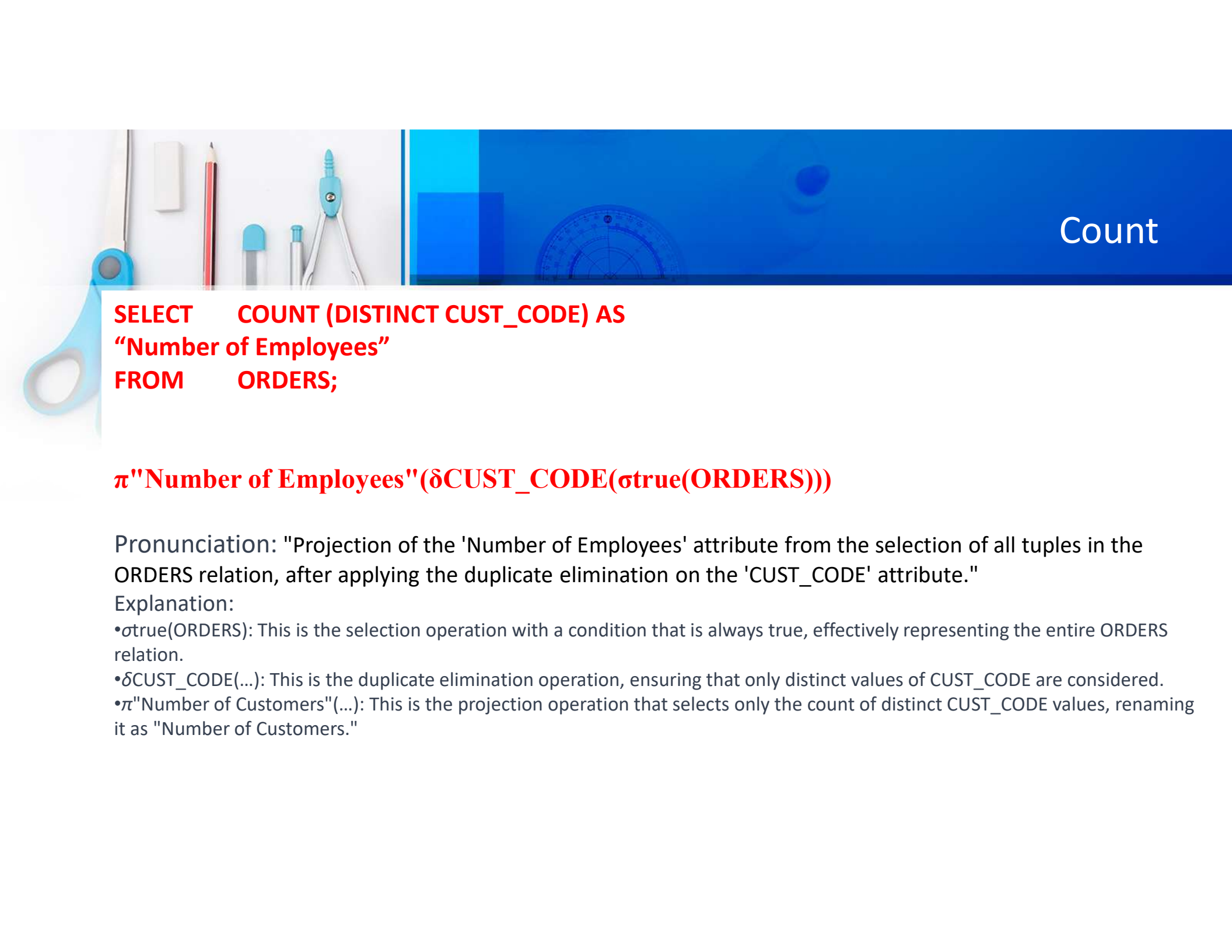
Week-11

Introduction to Database Systems

Shah Nawaz



Relational Algebra for Aggregate Functions and More



Count

```
SELECT    COUNT (DISTINCT CUST_CODE) AS  
"Number of Employees"  
FROM      ORDERS;
```

π "Number of Employees"(δ CUST_CODE(σ true(ORDERS)))

Pronunciation: "Projection of the 'Number of Employees' attribute from the selection of all tuples in the ORDERS relation, after applying the duplicate elimination on the 'CUST_CODE' attribute."

Explanation:

- σ true(ORDERS): This is the selection operation with a condition that is always true, effectively representing the entire ORDERS relation.
- δ CUST_CODE(...): This is the duplicate elimination operation, ensuring that only distinct values of CUST_CODE are considered.
- π "Number of Customers"(...): This is the projection operation that selects only the count of distinct CUST_CODE values, renaming it as "Number of Customers."



Group By

Example:

```
SELECT CustomerID, COUNT(*) AS OrderCount  
FROM Orders GROUP BY CustomerID;
```

```
 $\pi$ CustomerID, OrderCount( $\gamma$ CustomerID; COUNT(*) AS OrderCount(Orders))
```

Pronunciation: "Projection of 'CustomerID' and 'OrderCount' from the result of grouping Orders by 'CustomerID' and applying the COUNT aggregate function, with the result aliased as 'OrderCount'."

Explanation:

- γ CustomerID; COUNT(*) AS OrderCount(Orders): This is the grouping operation where we group the rows of the Orders table by CustomerID and calculate the count of rows (orders) for each group.
- π CustomerID, OrderCount(...): This is the projection operation that selects only the CustomerID and OrderCount columns from the result of the grouping operation.



Sum

Example:

```
SELECT SUM (Salary) AS TotalSalary FROM EMPLOYEE;
```

In relational algebra, the expression for the above SQL query can be represented using the γ (gamma) operator to perform the summation:

γ SUM(Salary) AS TotalSalary(EMPLOYEE)

Pronunciation: "Gamma: Sum of Salary as TotalSalary, applied to the EMPLOYEE relation."

Explanation:

- γ represents the Group By operation.
- "SUM(Salary) AS TotalSalary" inside γ indicates that we are performing the sum aggregation on the "Salary" attribute and renaming the result as "TotalSalary".
- The relation involved is "EMPLOYEE".



Max

Example:

SELECT MAX (Salary) AS MaximumSalary FROM EMPLOYEE;

The relational algebra expression for the above SQL query can be represented using the γ (gamma) operator to perform the maximum aggregation:

γ MAX(Salary) AS MaximumSalary(EMPLOYEE)

Pronounce: "Gamma: Maximum of Salary as MaximumSalary, applied to the EMPLOYEE relation."

Explanation:

- γ represents the Group By operation.
- "MAX(Salary) AS MaximumSalary" inside γ indicates that we are performing the maximum aggregation on the "Salary" attribute and renaming the result as "MaximumSalary".
- The relation involved is "EMPLOYEE".



Min

Example:

SELECT MIN (Salary) AS MinimumSalary FROM EMPLOYEE;

The relational algebra expression for the above SQL query can be represented using the γ (gamma) operator to perform the maximum aggregation:

γ MIN(Salary) AS MinimumSalary(EMPLOYEE)

Pronounce: "Gamma: Minimum of Salary as MinimumSalary, applied to the EMPLOYEE relation."

Explanation:

- γ represents the Group By operation.
- "MIN(Salary) AS MinimumSalary" inside γ indicates that we are performing the minimum aggregation on the "Salary" attribute and renaming the result as "MinimumSalary".
- The relation involved is "EMPLOYEE".

Example:

```
SELECT AVG (Salary) AS AverageSalary FROM EMPLOYEE;
```

The relational algebra expression for the above SQL query can be represented using the γ (gamma) operator to perform the maximum aggregation:

γ AVG(Salary) AS AverageSalary(EMPLOYEE)

Pronounce: "Gamma: Average of Salary as AverageSalary, applied to the EMPLOYEE relation."

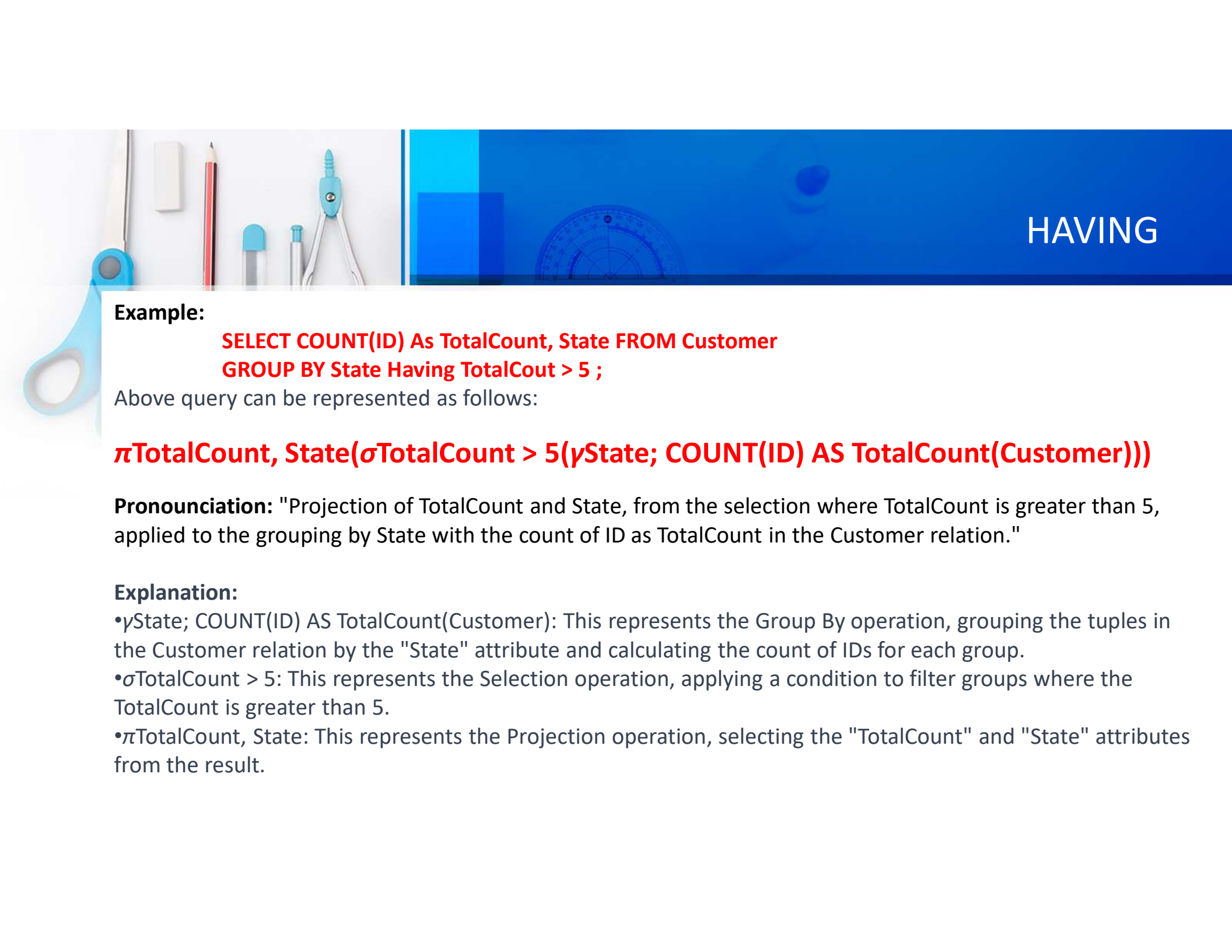
Explanation:

- γ represents the Group By operation.
- "MIN(Salary) AS AverageSalary" inside γ indicates that we are performing the Average aggregation on the "Salary" attribute and renaming the result as "AverageSalary".
- The relation involved is "EMPLOYEE".



HAVING

In relational algebra, there isn't a specific symbol dedicated to the HAVING clause, as relational algebra primarily focuses on basic operations like selection (σ), projection (π), join ($\bowtie$), and grouping (γ). The HAVING clause is a part of SQL and is used in combination with the GROUP BY clause to filter the results of a grouped query based on a specified condition.



HAVING

Example:

```
SELECT COUNT(ID) AS TotalCount, State FROM Customer  
GROUP BY State Having TotalCout > 5 ;
```

Above query can be represented as follows:

$\pi_{\text{TotalCount, State}}(\sigma_{\text{TotalCount} > 5}(\gamma_{\text{State; COUNT(ID) AS TotalCount}}(\text{Customer})))$

Pronunciation: "Projection of TotalCount and State, from the selection where TotalCount is greater than 5, applied to the grouping by State with the count of ID as TotalCount in the Customer relation."

Explanation:

- $\gamma_{\text{State; COUNT(ID) AS TotalCount}}(\text{Customer})$: This represents the Group By operation, grouping the tuples in the Customer relation by the "State" attribute and calculating the count of IDs for each group.
- $\sigma_{\text{TotalCount} > 5}$: This represents the Selection operation, applying a condition to filter groups where the TotalCount is greater than 5.
- $\pi_{\text{TotalCount, State}}$: This represents the Projection operation, selecting the "TotalCount" and "State" attributes from the result.

In relational algebra, the IN operator from SQL is typically expressed using a combination of operations. Let's consider a scenario where you want to retrieve all orders from the "Orders" relation where the customer ID is in a specified set {101, 102, 105}. The SQL query for this is:

SELECT * FROM Orders WHERE CustomerID IN (101, 102, 105);

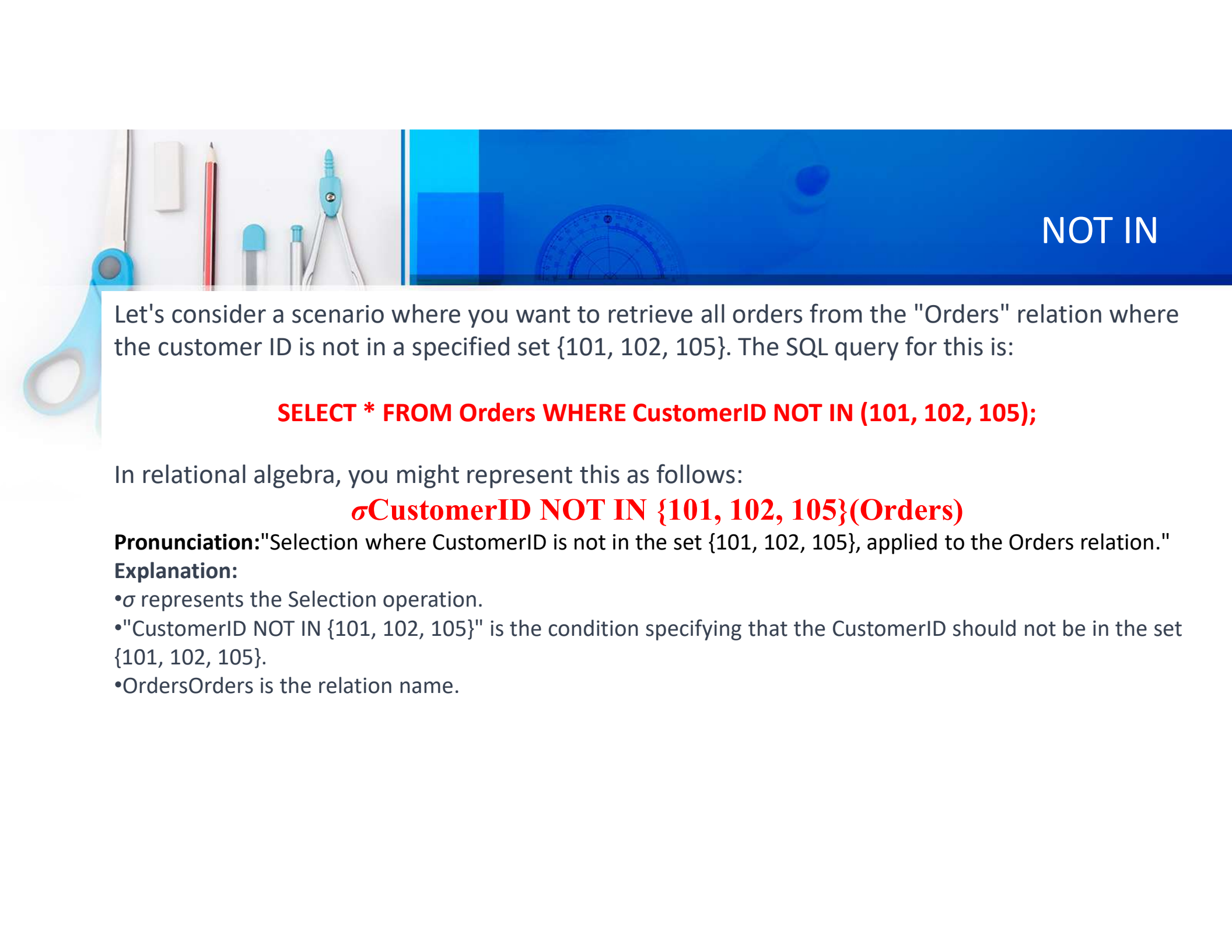
In relational algebra, you might represent this as follows:

$\sigma_{\text{CustomerID} \in \{101, 102, 105\}}(\text{Orders})$

Pronunciation: "Selection where CustomerID is in the set {101, 102, 105}, applied to the Orders relation."

Explanation:

- σ represents the Selection operation.
- " $\text{CustomerID} \in \{101, 102, 105\}$ " is the condition specifying that the CustomerID should be in the set {101, 102, 105}.
- Orders is the relation name.



NOT IN

Let's consider a scenario where you want to retrieve all orders from the "Orders" relation where the customer ID is not in a specified set {101, 102, 105}. The SQL query for this is:

```
SELECT * FROM Orders WHERE CustomerID NOT IN (101, 102, 105);
```

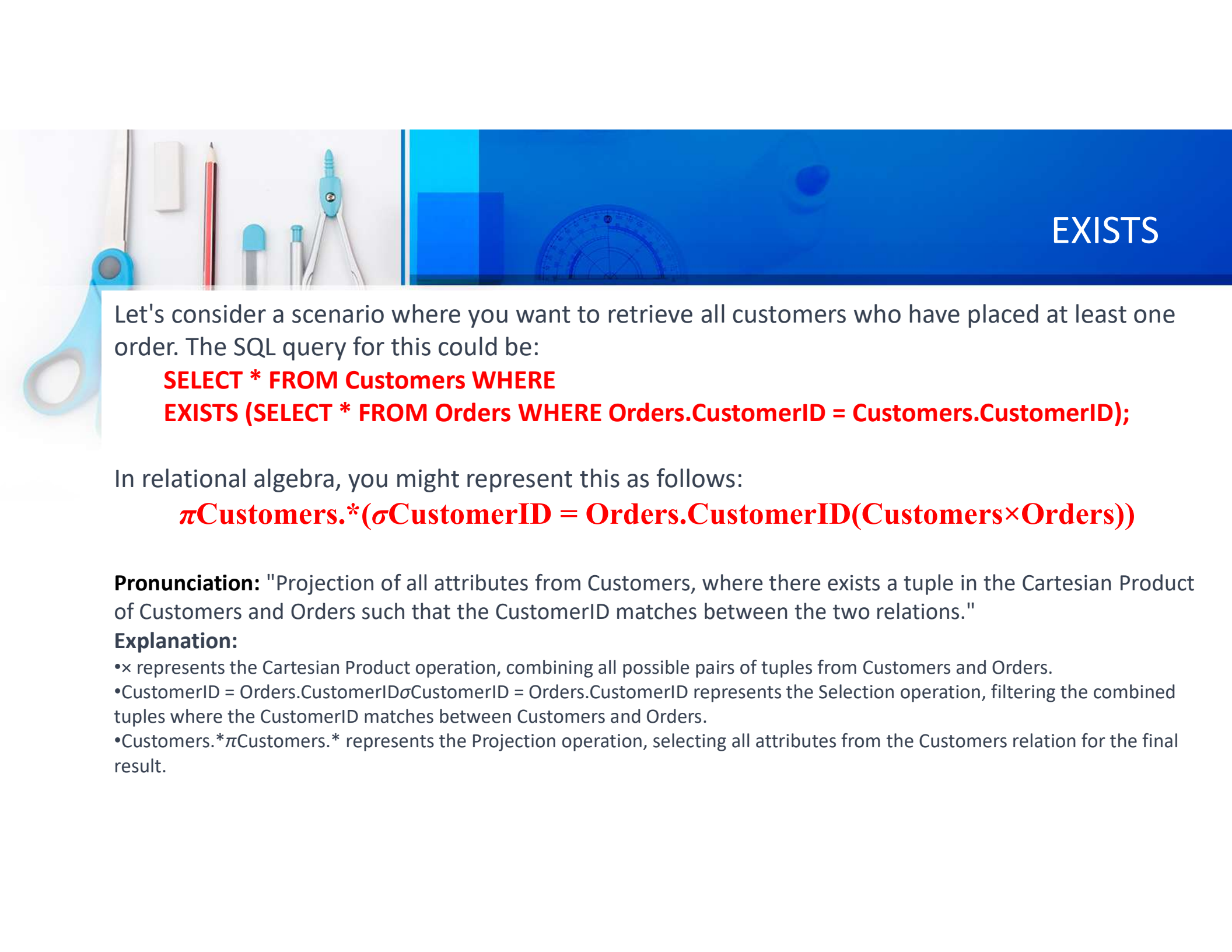
In relational algebra, you might represent this as follows:

```
 $\sigma_{\text{CustomerID NOT IN \{101, 102, 105\}}(\text{Orders})$ 
```

Pronunciation: "Selection where CustomerID is not in the set {101, 102, 105}, applied to the Orders relation."

Explanation:

- σ represents the Selection operation.
- "CustomerID NOT IN {101, 102, 105}" is the condition specifying that the CustomerID should not be in the set {101, 102, 105}.
- Orders is the relation name.



EXISTS

Let's consider a scenario where you want to retrieve all customers who have placed at least one order. The SQL query for this could be:

```
SELECT * FROM Customers WHERE  
EXISTS (SELECT * FROM Orders WHERE Orders.CustomerID = Customers.CustomerID);
```

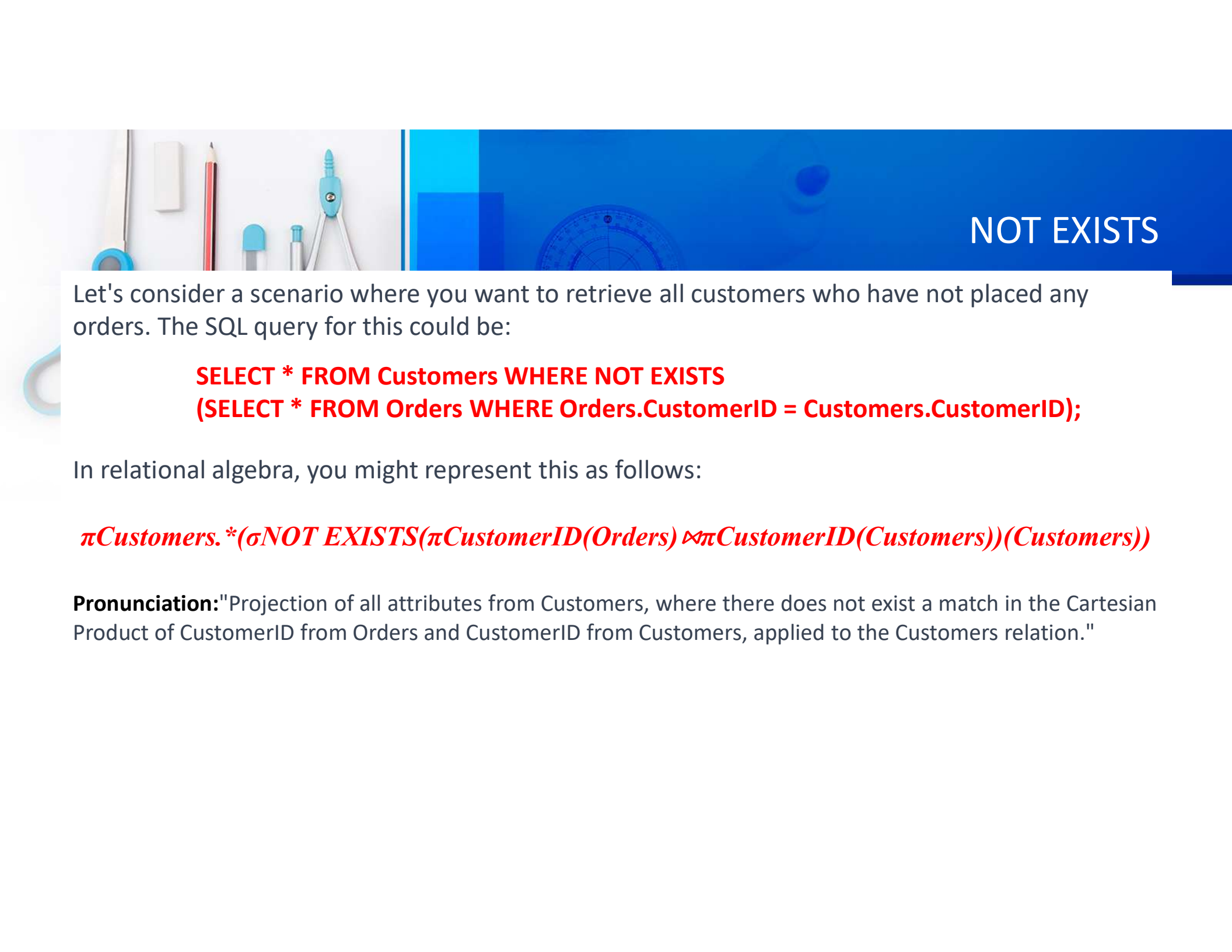
In relational algebra, you might represent this as follows:

```
 $\pi_{\text{Customers.}*}(\sigma_{\text{CustomerID} = \text{Orders.CustomerID}}(\text{Customers} \times \text{Orders}))$ 
```

Pronunciation: "Projection of all attributes from Customers, where there exists a tuple in the Cartesian Product of Customers and Orders such that the CustomerID matches between the two relations."

Explanation:

- $\times$ represents the Cartesian Product operation, combining all possible pairs of tuples from Customers and Orders.
- $\text{CustomerID} = \text{Orders.CustomerID}$ or $\sigma_{\text{CustomerID} = \text{Orders.CustomerID}}$ represents the Selection operation, filtering the combined tuples where the CustomerID matches between Customers and Orders.
- $\text{Customers.}* \pi_{\text{Customers.}*}$ represents the Projection operation, selecting all attributes from the Customers relation for the final result.



NOT EXISTS

Let's consider a scenario where you want to retrieve all customers who have not placed any orders. The SQL query for this could be:

```
SELECT * FROM Customers WHERE NOT EXISTS  
(SELECT * FROM Orders WHERE Orders.CustomerID = Customers.CustomerID);
```

In relational algebra, you might represent this as follows:

$\pi_{Customers} . * (\sigma_{NOT\ EXISTS} (\pi_{CustomerID}(Orders) \bowtie \pi_{CustomerID}(Customers)) (Customers))$

Pronunciation: "Projection of all attributes from Customers, where there does not exist a match in the Cartesian Product of CustomerID from Orders and CustomerID from Customers, applied to the Customers relation."



NOT EXISTS

Explanation:

Explanation:

- $\pi_{\text{CustomerID}}(\text{Orders})$ represents the Projection operation, selecting only the CustomerID attribute from the Orders relation.
- $\pi_{\text{CustomerID}}(\text{Customers})$ represents the Projection operation, selecting only the CustomerID attribute from the Customers relation.
- $\bowtie$ represents the Natural Join operation, combining all possible pairs of tuples from the two projections based on the common attribute CustomerID.
- $\text{NOT EXISTS}(\dots)$ represents the negation of the existence of tuples resulting from the join. If there is no match, the condition is satisfied.
- $\sigma_{\text{NOT EXISTS}(\dots)}$ represents the Selection operation with the NOT EXISTS condition.
- $*\pi_{\text{Customers}}.*$ represents the Projection operation, selecting all attributes from the Customers relation for the final result.



CREAT VIEW

Let's consider a scenario where you want to create a view named "HighValueOrders" that includes orders with a total amount greater than \$1000. The SQL query to create such a view might look like this:

```
CREATE VIEW HighValueOrders AS  
SELECT OrderID, CustomerID, OrderDate, TotalAmount FROM Orders  
WHERE TotalAmount > 1000;
```

In relational algebra, you might represent this as follows:

```
HighValueOrders ←  
πOrderID, CustomerID, OrderDate, TotalAmount(σTotalAmount > 1000(Orders))
```

Pronunciation: Create a view named HighValueOrders, which is the projection of OrderID, CustomerID, OrderDate, and TotalAmount from the selection of Orders where TotalAmount is greater than 1000."

The header features a light blue background on the left with various stationery items: a pair of scissors, an eraser, a pencil, a sharpener, and a compass. On the right, there is a dark blue gradient background with faint geometric shapes, including a square and a circle, and a semi-transparent ruler.

CREAT VIEW

Explanation:

- $\sigma_{\text{TotalAmount} > 1000}(\text{Orders})$ represents the Selection operation, filtering the tuples from the Orders relation where the TotalAmount is greater than \$1000.
- $\pi_{\text{OrderID, CustomerID, OrderDate, TotalAmount}}$ represents the Projection operation, selecting specific attributes from the result of the selection.
- $\text{HighValueOrders} \leftarrow$ represents the assignment of the result to a view named HighValueOrders.



UNION

Let's consider a scenario where you want to retrieve the distinct values of "City" from both the "Customers" and "Suppliers" tables using the UNION operator. The SQL query and its relational algebra expression might look like this:

SELECT City FROM Customers UNION SELECT City FROM Suppliers;

In relational algebra, you might represent this as follows:

$\pi_{\text{City}}(\text{Customers}) \cup \pi_{\text{City}}(\text{Suppliers})$

Pronunciation: "Projection of the 'City' attribute from the 'Customers' table unioned with the projection of the 'City' attribute from the 'Suppliers' table."

Explanation:

- $\pi_{\text{City}}(\text{Customers})$ represents the Projection operation, selecting the "City" attribute from the "Customers" relation.
- $\pi_{\text{City}}(\text{Suppliers})$ represents the Projection operation, selecting the "City" attribute from the "Suppliers" relation.
- $\cup$ represents the Union operation, combining the distinct values of "City" from both projections.



UNION ALL

Let's consider a scenario where you want to retrieve all values of "City" from both the "Customers" and "Suppliers" tables, including duplicates, using the UNION ALL operator. The SQL query and its relational algebra expression might look like this:

SELECT City FROM Customers UNION ALL SELECT City FROM Suppliers;

In relational algebra, you might represent this as follows:

$\pi_{\text{City}}(\text{Customers}) \cup \text{ALL } \pi_{\text{City}}(\text{Suppliers})$

Pronunciation: "Projection of the 'City' attribute from the 'Customers' table union all with the projection of the 'City' attribute from the 'Suppliers' table."

Explanation:

- $\pi_{\text{City}}(\text{Customers})$ represents the Projection operation, selecting the "City" attribute from the "Customers" relation.
- $\pi_{\text{City}}(\text{Suppliers})$ represents the Projection operation, selecting the "City" attribute from the "Suppliers" relation.
- $\cup \text{ALL}$ represents the Union All operation, combining all values of "City" from both projections, including duplicates.



INTERSECT

Let's consider a scenario where you want to retrieve the common values of "City" from both the "Customers" and "Suppliers" tables using the INTERSECT operator. The SQL query and its relational algebra expression might look like this:

SELECT City FROM Customers INTERSECT SELECT City FROM Suppliers;

In relational algebra, you might represent this as follows:

$\pi_{\text{City}}(\text{Customers}) \cap \pi_{\text{City}}(\text{Suppliers})$

Pronunciation: "Projection of the 'City' attribute from the 'Customers' table intersect with the projection of the 'City' attribute from the 'Suppliers' table."

Explanation:

- $\pi_{\text{City}}(\text{Customers})$ represents the Projection operation, selecting the "City" attribute from the "Customers" relation.
- $\pi_{\text{City}}(\text{Suppliers})$ represents the Projection operation, selecting the "City" attribute from the "Suppliers" relation.
- $\cap$ represents the Intersection operation, retrieving the common values of "City" from both projections.



Thank You all!